

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
Benyoucef BENKHEDDA University — Algiers 1



Faculty of Sciences
Computer Science Department

Final Year Project for the Bachelor's Degree in Computer Science

Option: Computer Systems and Software

Theme

Centralized Software Deployment Platform (CSDP)

Supervised by:

Mme Bisma Bekkai
Mr Ouali Laarbi

Directed by:

Djelouah Youcef
Messaouali Abdelhadi
Meziti Rayane Aymane

2025/2026

Acknowledgements

Before we begin with this report, we would like to acknowledge all those people that helped us so much and thereby enabled us to prepare this report.

Our sincere appreciation goes to the supervisors and the entire Information Systems Division at Algérie Télécom. We have no words to thank them; we simply want to convey our deep gratitude and respect to them.

God bless you and your family.

Dedications

To our dear parents

To our whole family

To everyone we care about

To all our friends(nassim in particualr)

This effort resulting from many years of studying

We dedicate the fruits of our labour

~ Djelouah Youcef, Messaouali Abdelhadi & Meziti Rayane Aymane ~

Abstract

The key purpose of this project is to develop and deploy a centralized software distribution platform for Algérie Télécom. The software provides features such as software package management, automated installation in client workstations, and tracking of the process status. An autonomous lightweight agent installed on each client workstation is responsible for detecting tasks, downloading packages, installing them, and reporting their status.

For the development of our software platform, we made use of UML modeling, Node.js and Express.js using TypeScript as the back-end framework, React for the administrative interface, and Bash for the client agent.

Keywords: Software distribution, Automated deployment, Client agent, Algérie Télécom, Node.js, React, TypeScript.

Résumé

Ce projet a été réalisé au sein d'Algérie Télécom pour remplacer le processus existant, qui est entièrement manuel, par une plateforme automatisée. Ce dernier processus impliquait des mises à jour effectuées manuellement, sans contrôle de version, intégrité, et détection des erreurs de déploiement.

La solution proposée est l'Automated Download Platform, qui utilise trois composants principaux : le frontend React/Vite, la stack backend Node.js/Express avec la base de données SQLite, ainsi qu'un agent daemon Bash installé sur chaque poste client. L'agent fait une requête au serveur chaque 30 secondes, installe silencieusement les paquets distribués et rapporte le succès du déploiement en temps réel. SHA256 sert à vérifier l'intégrité des paquets, tandis que JWT fournit l'accès administrateur.

Contents

Acknowledgements	i
Dedications	ii
Abstract / Résumé	iii
List of Figures	viii
List of Tables	viii
List of Abbreviations	ix
General Introduction	1
1 General Context Presentation	2
1.1 Introduction	2
1.2 Presentation of the Host Organization	2
1.2.1 Algérie Télécom — Overview	2
1.2.2 Historical Background	3
1.2.3 Activities and Services	4
1.2.4 Organizational Structure	4
1.3 Study of the Existing System	5
1.3.1 Description of the Existing Procedures	5
1.3.1.1 Identification of Software Version	5
1.3.1.2 Package Preparation and Distribution	5
1.3.1.3 Deployment Tracking and Reporting	6
1.3.1.4 Update Verification and Compliance	6
1.3.2 Identified Problems and Weaknesses	6
1.3.3 Project Objectives	7
1.3.3.1 Summary of Key Benefits	8
1.4 Conclusion	9
2 Modeling and Design	10
2.1 Introduction	10

2.2	Selection of Design Methodology	10
2.2.1	The UML Language	10
2.2.2	Working Environment	11
2.3	Use Case Diagram	11
2.3.1	Definition	11
2.3.2	Requirements Specification	11
2.3.2.1	Functional Requirements	11
2.3.2.2	Non-Functional Requirements	12
2.3.3	Identification of Actors and Their Use Cases	12
2.3.4	General Diagram	12
2.4	Sequence Diagrams	13
2.4.1	Definition	13
2.4.2	Authentication	14
2.4.3	Upload a Software Package	16
2.4.4	Create a Deployment	18
2.4.5	Agent Heartbeat & Task Execution	20
2.4.6	Agent Registration	22
2.5	Class Diagram	23
2.5.1	Definition	23
2.6	Relational Model	24
2.7	Conclusion	25
3	Implementation	26
3.1	Introduction	26
3.2	Development Environment	26
3.2.1	Programming Languages	26
3.2.1.1	TypeScript / JavaScript	26
3.2.1.2	Bash (Shell Scripting)	26
3.2.1.3	SQL (SQLite)	27
3.2.2	Frameworks and Libraries	27
3.2.2.1	React + Vite	27
3.2.2.2	Express.js	27
3.2.2.3	TypeORM	27
3.2.3	Development Tools	28
3.2.3.1	Node.js	28
3.2.3.2	Visual Studio Code	28
3.2.3.3	Additional Libraries	28
3.3	Application Interfaces	28
3.3.1	Authentication Interface	28

3.3.2	Main Dashboard	30
3.3.3	Package Management Interface	31
3.3.3.1	Package List	31
3.3.3.2	Upload a New Package	32
3.3.3.3	Package Details	33
3.3.4	Deployment Management Interface	34
3.3.4.1	Create a Deployment	34
3.3.4.2	Monitor Deployment Progress	36
3.3.5	Client Management Interface	37
3.3.5.1	Registered Clients List	37
3.3.5.2	Client Detail View	38
3.3.6	Agent Registration (First-Time Setup)	39
3.4	Conclusion	40
General Conclusion and Perspectives		41
References / Bibliography		42

List of Figures

1.1	Algerie Telecom's Logo	3
2.1	General Use Case Diagram of the Automated Download Platform . . .	13
2.2	Sequence Diagram — Authentication	15
2.3	Sequence Diagram — Upload a Software Package	17
2.4	Sequence Diagram — Create a Deployment	19
2.5	Sequence Diagram — Agent Heartbeat & Task Execution	21
2.6	Sequence Diagram — Agent Registration	23
2.7	Class Diagram of the Automated Download Platform	24
3.1	Authentication Interface	29
3.2	Invalid Authentication — Error Message	30
3.3	Main Dashboard Interface	31
3.4	Package Management — Package List	32
3.5	New Package Upload Form	33
3.6	Package Detail View with Version History	34
3.7	Deployment Form — Select Package	35

3.8	Deployment Form — Select Version	35
3.9	Deployment Form — Select Client	36
3.10	Deployment Form — Confirmation	36
3.11	Deployment Monitor — Per-Client Status	37
3.12	Registered Client Machines Interface	38
3.13	Client Detail — Installed Packages	39
3.14	Agent Registration — Terminal Output	39

List of Tables

1.1	Algérie Télécom — Company Information	3
2.1	Actors and Their Roles	12
2.2	Textual Description — Authentication	14
2.3	Textual Description — Upload a Software Package	16
2.4	Textual Description — Create a Deployment	18
2.5	Textual Description — Agent Heartbeat & Task Execution	20
2.6	Textual Description — Agent Registration	22

List of Abbreviations

Abbreviation	Full Form
ADSL	Asymmetric Digital Subscriber Line
API	Application Programming Interface
ARPT	Autorité de Régulation de la Poste et des Télécommunications
AT	Algérie Télécom
DSI	Direction des Systèmes d'Information
FTTH	Fiber To The Home
HTTP	HyperText Transfer Protocol
ICT	Information and Communications Technology
IHM	Interface Homme-Machine
IT	Information Technology
JWT	JSON Web Token
MPTIC	Ministère de la Poste, des Télécommunications, des Technologies et du Numérique
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études
REST	Representational State Transfer
SHA256	Secure Hash Algorithm 256-bit
SPA	Société Par Actions
SQL	Structured Query Language
UML	Unified Modeling Language
UUID	Universally Unique Identifier
VDSL	Very-high-speed Digital Subscriber Line
VPN	Virtual Private Network
VS Code	Visual Studio Code
WWW	World Wide Web

General Introduction

The information technology infrastructure of modern organizations requires efficient management of the software deployed across all workstations. Manual update procedures are error-prone, time-consuming, and do not scale to environments with hundreds of client machines spread across multiple geographic locations.

This project was carried out within Algérie Télécom, Algeria's national public telecommunications company, with the objective of replacing the existing fully manual software distribution process with a centralized, automated platform. The result is the **Automated Download Platform**, a three-tier system composed of a React/Vite web front-end for administration, a Node.js/Express/TypeORM back-end API, and a lightweight Bash agent daemon running on every client machine.

This report is organized as follows:

- **Chapter 1** presents the host organization and analyzes the existing manual process, identifying its weaknesses and formulating the project objectives.
- **Chapter 2** describes the system analysis and UML design, including use-case diagrams, sequence diagrams, the class diagram, and the relational model.
- **Chapter 3** covers the implementation phase, presenting the development environment, technologies used, and the application interfaces.

The report closes with a general conclusion summarizing the outcomes of the project and outlining possible future improvements.

Chapter 1

General Context Presentation

1.1 Introduction

This chapter provides a background to the environment in which our final year project was conceived and developed. It starts by giving a brief introduction to our host company, Algérie Télécom, the national public telecom company of Algeria, whose operational difficulties necessitated the development of a robust software package distribution and update system.

Next, we analyze the current status of software deployment in the organization. This includes highlighting the existing manual process, analyzing the shortcomings of this process, and establishing the goals that guided the design of our software solution — the Automated Download Platform, an automated system for distributing, signing, publishing, and deploying software packages and updates across the entire network.

The software distribution solution is implemented using a client-server architecture comprising three major components. The first component is the central server, which handles all the operations related to storing, signing, and publishing the software packages. The second component is the client-side program, which is a small footprint program running on each computer in the network. It detects available updates, downloads, verifies, and installs software updates in a silent fashion.

1.2 Presentation of the Host Organization

1.2.1 Algérie Télécom — Overview

Algérie Télécom (AT) is the state-owned public telecommunications provider for the country of Algeria. It is a public economic entity founded on January 1st, 2003, after the privatization of Algeria's telecommunications sector and the split between the postal services and telecommunications from Algérie Poste. Algérie Télécom offers a wide variety of voice, data, and internet services for millions of citizens, organizations, and governmental entities.[3]

The company has its headquarters in Algiers and maintains a telecommunications infrastructure covering the entire country using technologies like fiber-optic backbones, copper access network, mobile infrastructure provided by its subsidiary Mobilis, and satellite coverage for remote areas. Algérie Télécom is supervised by the Ministry of Post and Telecommunications.



Figure 1.1: Algérie Telecom’s Logo

Table 1.1: Algérie Télécom — Company Information

Legal name	Algérie Télécom — Société par Actions (S.P.A)
Founded	January 1, 2003
Headquarters	Route Nationale No. 5, Cinq Maisons, Mohammadia, Algiers
Sector	Telecommunications — Public Economic Enterprise
Services	Fixed telephony, ADSL/FTTH internet, leased lines, data services
Subsidiary	Mobilis (mobile telecommunications)
Supervision	Ministry of Post and Telecommunications (MPTIC) [4]

1.2.2 Historical Background

History of Algérie Télécom is linked with the overall history of the development of the Algerian telecommunication sector. For quite a long period, provision of telecommunication services was handled by the state through the Ministry of Post and Telecommunications in Algeria. However, during the decade of the 1990s when information and telecommunication technologies were developing fast alongside the growing demand for telecommunication services in society, the need for structural reforms became obvious.

In 2000, the Algerian authorities adopted Law No. 2000-03 on August 5 [1], 2000. The law was aimed at regulating and liberalizing the telecommunications sector. The law established ARPT [2] and provided for separation of postal services and telecommunications. In turn, on January 1, 2003, the entity Algérie Télécom was officially registered as a public economic company. Algérie Télécom became the successor to all properties,

including infrastructure, human resources, and technological expertise of the former Directorate of Telecommunications.

Since its inception, Algérie Télécom has been implementing large-scale projects related to infrastructure development. They included introduction of the Internet service based on the ADSL technology, FTTH in cities and suburban areas, and upgrades of the backbone network. At the moment, millions of customers use fixed-line telephone services and access the Internet via the services provided by Algérie Télécom while its subsidiary Mobilis provides one of the best mobile network services in Algeria.

1.2.3 Activities and Services

Algérie Télécom manages various key service lines in order to ensure coverage of all the telecommunications service needs of modern society:

- **Telephony service:** management and maintenance of the fixed telephone network in Algeria, which serves both residential and business customers by providing telephony services.
- **Internet services:** ADSL and VDSL services delivered via the copper network as well as fiber-optic Internet services (FTTH) in suitable regions.
- **Dedicated lines and enterprise solutions:** dedicated connections, Virtual Private Network services (VPN) and other business-class telecom services.
- **Data Center Services:** co-location, cloud hosting and other ICT services provided from the company's Data Centers.
- **Satellite services:** satellite-based telecommunication services that enable connectivity in remote and rural regions with no land-based network.
- **Mobile services (Mobilis brand):** voice and data services, mobile payment services, and various value added services provided under the brand of its wholly owned subsidiary Mobilis.

In order to deliver these different types of service, Algérie Télécom has developed an extensive organizational structure consisting of regional offices, local centers and technical units located in all 58 wilayas of Algeria.

1.2.4 Organizational Structure

The structure of Algérie Télécom is built upon a headquarter that includes the departments of corporate governance, strategy, finance, and human resources. Under the headquarter level, the company operates regional directorates (*Directions Régionales*)

charged with the responsibility of ensuring the efficient delivery of services, management of the infrastructure, and customer relationships in a specific geographic area. Every regional directorate is broken down into smaller units at district level.

The corporate structure of the company includes specific divisions dealing with the issues of network operations, information systems, commerce, and technical support. In particular, the division in charge of managing the software of the enterprise and its applications is the Information Systems Division (*Direction des Systèmes d'Information* — DSI). The necessity of deploying and supporting software at hundreds of workstations in different locations prompted the creation of the Automated Download Platform.

1.3 Study of the Existing System

1.3.1 Description of the Existing Procedures

Prior to introducing the Download Center solution, the existing software management processes at Algérie Télécom relied largely on manual activities. The description below outlines all stages in the software management procedure currently in use, starting from detection of the necessity for a new version of the software installed or applied to end user PCs.

1.3.1.1 Identification of Software Version

If a new version of the software, patch or other updates become available (either from an external source or produced internally), this must be approved first by IT specialists. This process takes place through the manual analysis of release notes, internal testing in the lab and approval of this decision from the respective IT manager.

This approval process takes a lot of time since it involves emailing, sending memos, and signing off on physical copies, which may last from several days up to a couple of weeks. There are no systems for keeping track of the status of software evaluation, and there is no system for automatic alerts about the availability of the next software version.

1.3.1.2 Package Preparation and Distribution

IT technicians manually create and distribute the installation packages of the software after its approval for deployment. It means that installation packages are manually copied to a USB drive or any available network location and physically deployed by IT personnel to targeted workstations, or deployed to workstations by means of remote

desktop connections established on-the-go. When updating numerous workstations, IT personnel have to perform installation procedures manually one by one.

In case there are no distribution servers centrally available to manage software updates, there is no standard approach to create, sign, and validate integrity of software packages prior to their deployment. IT technicians have to manually determine the appropriate version of a package to be installed and make sure they choose right installation files.

1.3.1.3 Deployment Tracking and Reporting

After a successful deployment of software packages, IT technicians manually log down installed software version and machines where the installations were performed by filling up spreadsheets stored locally on their computers. These logs vary from department to department; while some departments have elaborate reports tracking deployed versions of software packages, others do not have any records of software deployments at all. The company lacks any unified reporting system to monitor software deployments.

If discrepancies between logs of software deployed versions occur, for example, there are out-of-date or unapproved versions installed, troubleshooting these issues is done manually.

1.3.1.4 Update Verification and Compliance

It remains a consistent challenge within the existing framework to ensure that all computers operate the approved versions of the enterprise software that is up to date. There is no automated tool available for IT managers to determine the software version on their computers. The auditing process happens sporadically and involves human effort, resulting in the discovery that a set of computers had not received updates recently.

1.3.2 Identified Problems and Weaknesses

The review of the existing software management process highlights several critical flaws that hinder the efficient, reliable, and safe operation of IT infrastructure at Algérie Télécom:

- **Demanding manual and non-scalable update procedure:** Every software update has to be conducted manually by the IT department employees through either on-site access to each machine or a remote desktop connection to the workstation in question. This method is inefficient and unsustainable in an environment where there are hundreds of workstations located in various regional departments.

- **Lack of a central repository:** There is no dedicated repository of approved software packages hosted and managed by the organization. Instead, software installations come via USB-sticks and shared drives without any specific version information attached.
- **Lack of automated software update alerts on client workstations:** Currently, there is no way for the user or the employee responsible for local IT services to be informed that the currently installed version of a particular application is not the newest and approved one. Users and local IT have to rely on the information provided by the central IT department.
- **No centralized software deployment tracking capability:** Currently, it is impossible for the organization to have a unified, real-time view of all the software packages installed and their respective versions. Tracking software deployments is currently only possible through various spreadsheet documents hosted by individual workgroups.
- **Mismatch of software versions:** Inefficient software update procedure leads to the situation where there are many instances of various software products with different versions among company computers, thus creating a risk of software conflicts and compatibility issues.
- **Lack of software integrity protection mechanisms:** Currently, software packages delivered are not secured in any way. Employees have no ability to validate the authenticity of any given package and detect any signs of possible corruption or manipulation during the distribution phase.
- **High staff burden and opportunity cost:** Time invested by IT personnel in manually updating the software is an opportunity cost. Efforts which could have been used more effectively to enhance IT services like network management and improvement can now only be applied to the less valuable activity of software deployment.
- **Lack of documentation and transfer of knowledge:** Since there is no systemized document available for reference, any departure from the company of IT personnel leaves gaps in organizational knowledge regarding software, thus making it difficult for new IT personnel to grasp the situation.

1.3.3 Project Objectives

The Download Center Platform for Algérie Télécom strives to achieve the following objectives:

To centralize software distribution: Give the administrator access to a web inter-

face through which he can distribute software packages to multiple client machines.

To enable software installation from the network automatically: The software installation will be automated by pushing software to client machines through an agent daemon that executes in each machine's background.

To track each deployment's progress and state (real-time): Monitor the different steps in the execution of a task (e.g., pending $\Rightarrow$ downloading $\Rightarrow$ installing $\Rightarrow$ success/failure) for each client machine.

To ensure software package integrity: Packages will be authenticated via SHA256 checksums and passwords will be authenticated using BCrypt.[13]

To maintain a comprehensive audit log: Every important action done by users and the system (registration, installation, failed tasks) will be logged.

To monitor client availability: Automatically determine if a client machine is offline and update its state accordingly.

1.3.3.1 Summary of Key Benefits

The benefits that the Automated Download Platform will bring to Algérie Télécom include:

- A substantial decrease in time and labor needed to install the updates throughout the entire company.
- Ending the problem of version discrepancies through implementation of a reliable system of enforcing officially approved versions.
- Better security through the requirement of package signing and control over the distribution channels.
- The ability to track down any software installations and ensure their auditability.
- Limitation of possible human error in the installation process through automation.
- Scalability of the proposed architecture in terms of the growing software library and expanding number of managed machines.

Solutions:

In order to address the challenges presented by this manual process, we created the **Automated Download Platform** — a centralized software distribution tool integrated into Algérie Télécom's internal network infrastructure.

This tool enables IT managers to download software packages to all computers at once using one easy-to-use web-based interface, requiring no physical presence on-site nor any remote desktop connections whatsoever. The moment a package is uploaded and the deployment task is initiated, the silent process agent located on each of the client machines performs all the heavy lifting automatically: downloads the file, installs it, and sends feedback to the server.

At the same time, our platform makes sure that all downloaded packages are authentic and untampered with, maintains an audit trail of all downloads, and gives IT managers insight into the status of every workstation along with the inventory of all software installed on them.

Thus, we managed to create a seamless automated software distribution pipeline that saves a considerable amount of IT resources and guarantees version consistency across the company.

1.4 Conclusion

This chapter has provided a general overview of the background of our engineering project in our last year of study. This chapter has introduced Algérie Télécom, the national telecommunication provider in Algeria, and has described the organizational context within which the project was executed. After that, a detailed analysis of the current process used for managing and updating the software was done, where we pinpointed the major drawbacks of the current process, which relies heavily on a manual approach to performing its tasks. The most notable drawback of the current process is the lack of centralized package management,[10] the inability to detect updates automatically, mismatching versions, and the lack of visibility and traceability during the update procedure.

Based on the previous analysis, we have formulated our objectives for designing the Automated Download Platform system, which consists of a server-client structure, consisting of a central package manager with digital signature capability, an update client capable of detecting and applying software updates autonomously, and a web-based admin panel that allows monitoring and management of the update process in real time.

The next chapters will deal with the design of the system, starting with Chapter 2, where we analyze the system requirements, the use-case diagram, and the complete system architecture.

Chapter 2

Modeling and Design

2.1 Introduction

Modeling is an approach that converts the problem's solution into a modeled diagram. The following section will describe the design and modeling process of our Automated Download Platform. This step is vital in ensuring that one can comprehend the suggested solution and fully appreciate the workings of the new platform, its features, and get ready for the implementation phase.

2.2 Selection of Design Methodology

In this project, UML will be used since it effectively captures both the static and dynamic perspectives of the information system. It is important to carry out a detailed analysis in order to determine the development requirements and some execution scenarios.

2.2.1 The UML Language

UML (Unified Modeling Language) is the language enabling graphical modeling of the solution of a practical problem through diagrams. In case the system is too complicated to understand, design, and build in a single attempt, it is wise to adopt an iterative approach. For this reason, we decided to use the spiral lifecycle approach because it is generic.[5]

The following figure represents the major phases in this software development life cycle:

- **Requirements analysis:** seeks to understand the objectives, alternatives, and constraints.
- **Design:** consists of organizing the code into various modules.
- **Development:** executes the selected solution.
- **Evaluation:** evaluates the outcomes and the next spiral iteration.[6]

2.2.2 Working Environment

The decision to go for StarUML to make our diagrams was made because StarUML is a very powerful UML diagramming application, which offers full support for all the UML diagrams needed by us for our project. StarUML offers a very user-friendly interface, as well as allows us to create professional-looking diagrams like the use case diagram, class diagram, sequence diagram, activity diagram, and deployment diagram. The fact that we can export diagrams from StarUML in PDF, PNG, SVG, and HTML documentation formats adds more weight to our choice.

2.3 Use Case Diagram

2.3.1 Definition

This is the identification of functionalities and interactions (use cases) between the actor and the future system. We can model it to:

- Understand the flow of the requested system.
- Identify actors and their use cases.
- Model the use case diagram with relationships.

2.3.2 Requirements Specification

Requirement gathering involves identifying the needs that will be fulfilled by the application.

2.3.2.1 Functional Requirements

The system shall allow only authorized users to execute the following actions:

1. User management by Administrator (create, update, delete, view).
2. System configuration management and logging viewing by Administrator.
3. Software package uploading and version management by Administrator.
4. Deployment creation and client selection by Administrator.
5. Agent registration (client PC) with server.
6. Heartbeat sending by Agent (every 30 seconds).
7. Pending deployment task retrieval by Agent.
8. Software package downloading by Agent.

9. Software package installation and success/failure reporting by Agent.[12]

2.3.2.2 Non-Functional Requirements

- Provide a clearly laid-out web interface to support the deployment process (quality).
- Make sure that the software is portable to all software platforms (Windows, Mac OS, and Linux).
- Conform to all ergonomic standards.
- Authenticate access to ensure security.

2.3.3 Identification of Actors and Their Use Cases

Table 2.1: Actors and Their Roles

Actor	Description
Administrator	Manages packages, creates deployments, monitors progress, consults logs, manages settings
Agent (Client PC)	Registers with server, sends heartbeat, downloads and installs packages, reports status
System	Stores files, updates statistics, generates logs, assigns tasks

2.3.4 General Diagram

The figure below represents the general use case diagram of the Download Center Platform. The three primary actors that have been recognized in this platform are: Administrator (user interacting with the website interface), Agent Daemon (automated background process for each individual client), and Heartbeat Monitor (server-side process checking the availability of clients).

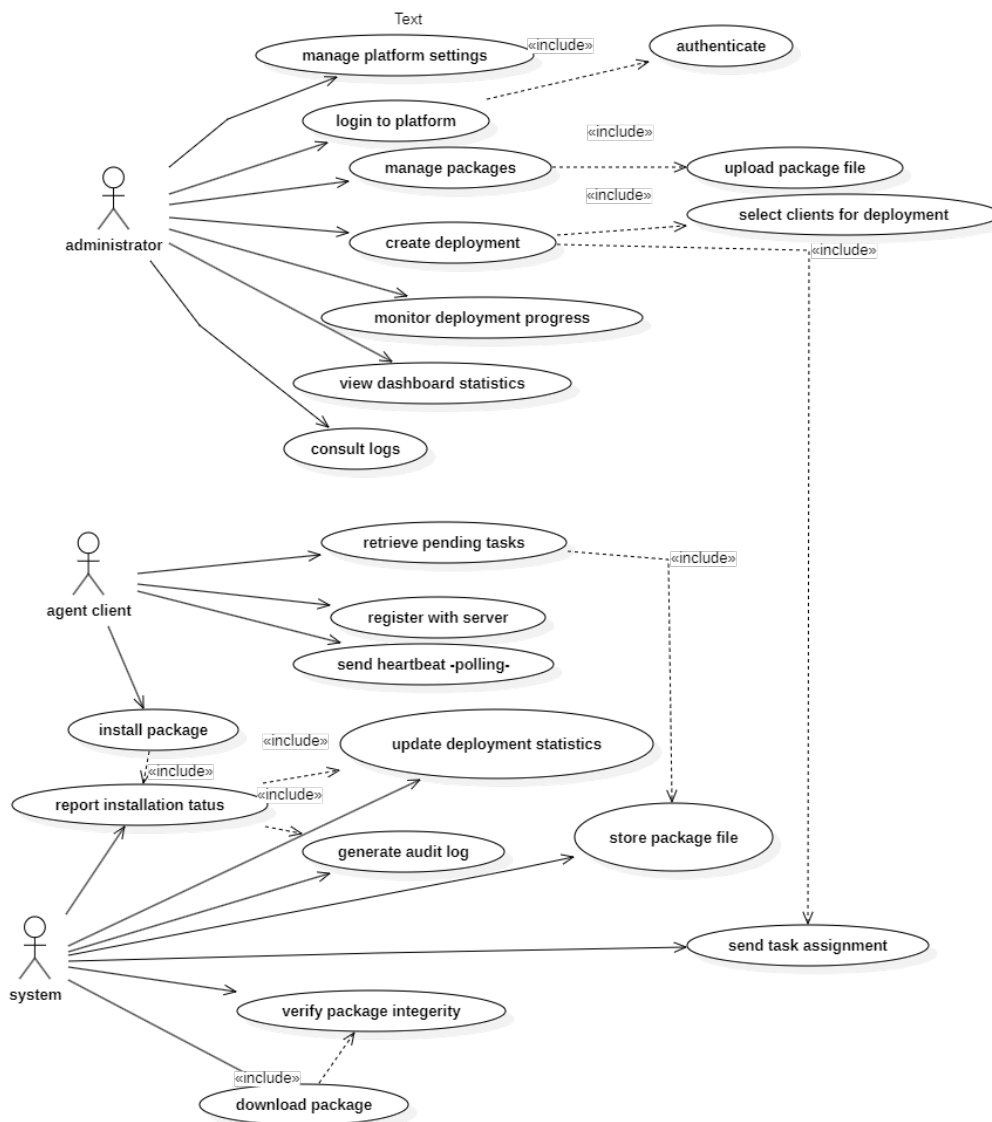


Figure 2.1: General Use Case Diagram of the Automated Download Platform

2.4 Sequence Diagrams

2.4.1 Definition

Sequence diagrams belong to the family of UML diagrams, which describe the interaction among various components within the system. The diagram highlights the message exchange pattern among objects (actors) in a certain order to execute the intended task or use case.

Unlike a class diagram, which depicts structure, a sequence diagram illustrates the behavior of the application by demonstrating interactions between objects or actors, including who communicates with whom and the sequence of the communication process.

Each primary use case is accompanied by a textual description and its respective UML sequence diagram.

2.4.2 Authentication

Table 2.2: Textual Description — Authentication

USE CASE	Authentication
OBJECTIVE	Access the Automated Download Platform administration interface.
PRIMARY ACTOR	Administrator.
SECONDARY ACTOR	System (Express API, SQLite database).
PRE-CONDITION	None.
NOMINAL SCENARIO	1. The administrator navigates to the login page. 2. The administrator enters their email address and password. 3. The system verifies the credentials against the User table (bcrypt comparison). 4. If valid, the system generates a JWT token and returns it along with the user profile. 5. The administrator is redirected to the dashboard.
ALTERNATIVE SCENARIO	3.1) If the email does not exist or the password is incorrect, the system returns HTTP 401 and displays the message: “Invalid credentials”.

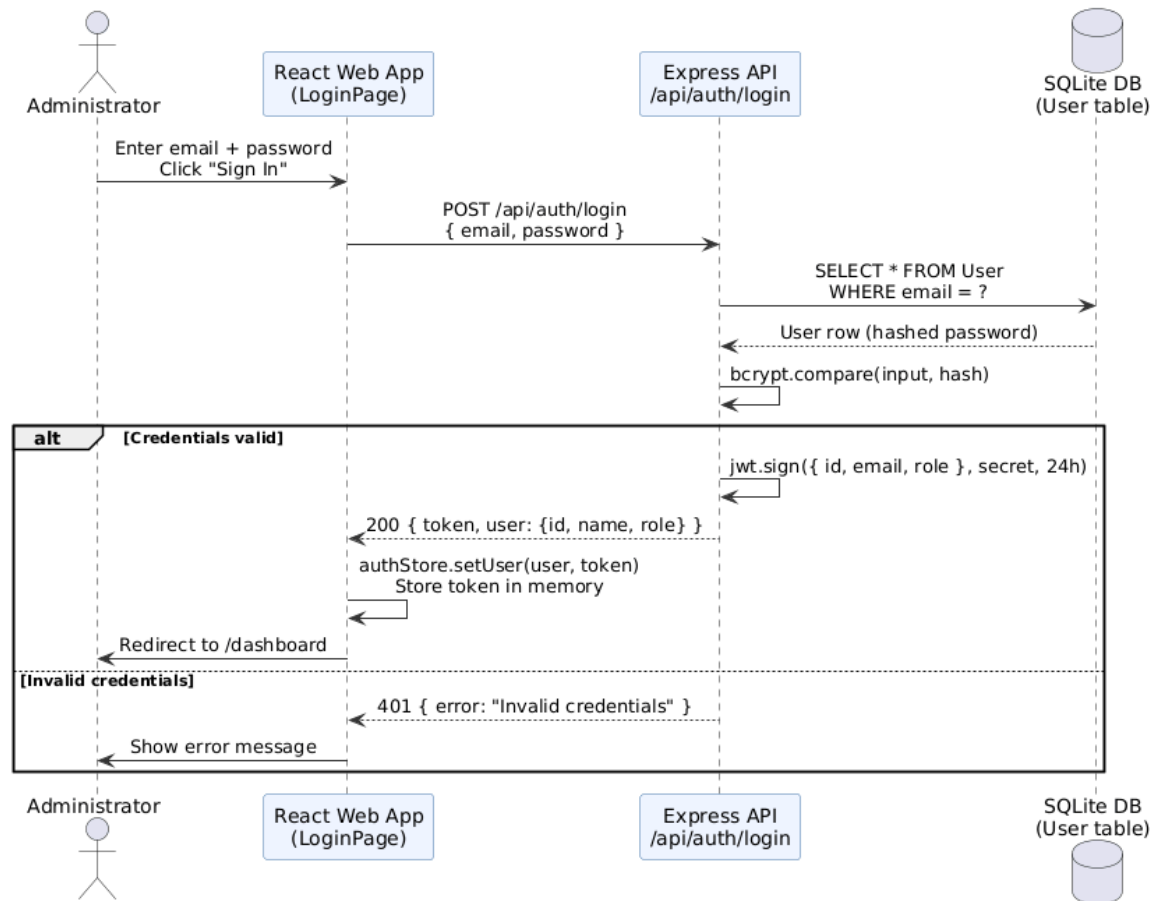


Figure 2.2: Sequence Diagram — Authentication

2.4.3 Upload a Software Package

Table 2.3: Textual Description — Upload a Software Package

USE CASE	Upload a Software Package
OBJECTIVE	Add a new software package (or a new version of an existing one) to the platform.
PRIMARY ACTOR	Administrator.
SECONDARY ACTOR	System (Multer file handler, File System, SQLite database).
PRE-CONDITION	Administrator is authenticated.
NOMINAL SCENARIO	Step 1: Administrator accesses the New Package page. Step 2: Administrator completes the form (name, description, version, architectures, and selects ZIP file). Step 3: Administrator submits the form. Step 4: System receives multipart request; uploads the file to a temporary folder. Step 5: System generates a SHA-256 hash of the uploaded file. Step 6: System copies the file to <code>uploads/<packageName>/</code>. Step 7: System creates a SoftwarePackage database object and a PackageVersion database object. Step 8: System responds with HTTP 201 and redirects administrator to the package detail page.
ALTERNATIVE SCENARIO	2.1) If a required field is missing, the system displays a validation error. 4.1) If the file exceeds 500 MB, Multer rejects the request with HTTP 400.

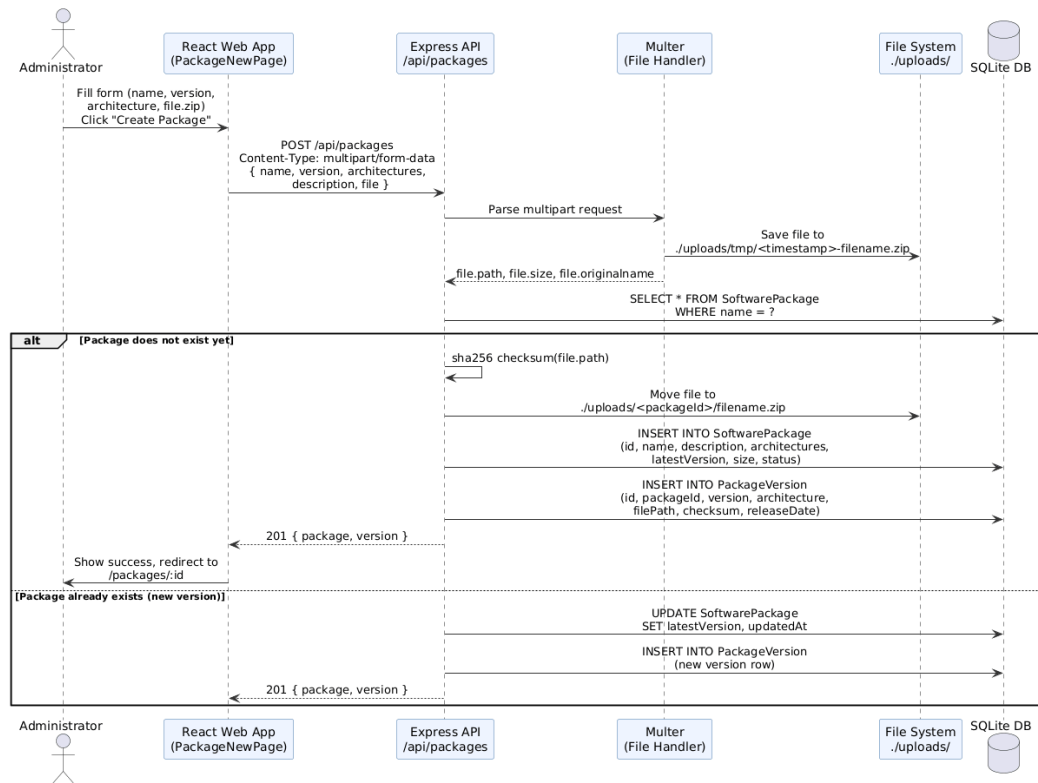


Figure 2.3: Sequence Diagram — Upload a Software Package

2.4.4 Create a Deployment

Table 2.4: Textual Description — Create a Deployment

USE CASE	Create a Deployment
OBJECTIVE	Push a software package version to one or more registered client machines.
PRIMARY ACTOR	Administrator.
SECONDARY ACTOR	System (Express API, SQLite database).
PRE-CONDITION	Administrator is authenticated. At least one package version and one registered client exist.
NOMINAL SCENARIO	Step 1: Administrator accesses the New Deployment page. Step 2: Administrator chooses the package, its version, and the target client(s). Step 3: Administrator clicks on the Deploy button. Step 4: System checks whether the package and version exist. Step 5: System creates a single Deployment object with status = Pending. Step 6: For each selected target, the system creates a DeploymentClient object with status = Pending. Step 7: System logs the deployment scheduled event for each target. Step 8: System responds with HTTP 201 and redirects the browser to the deployment detail page.
ALTERNATIVE SCENARIO	2.1: If no client is chosen, the system returns a validation error. 4.1: If either the package or the version doesn't exist, the system returns HTTP 404.

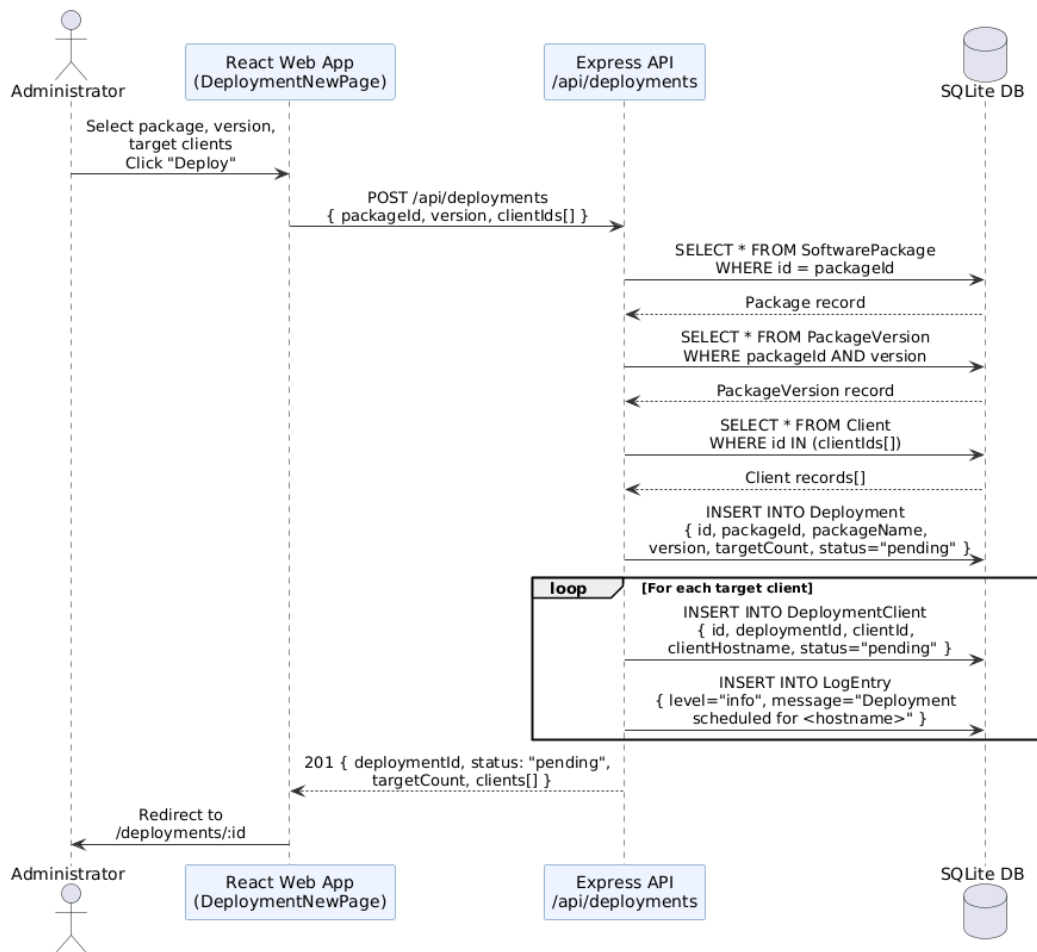


Figure 2.4: Sequence Diagram — Create a Deployment

2.4.5 Agent Heartbeat & Task Execution

Table 2.5: Textual Description — Agent Heartbeat & Task Execution

USE CASE	Agent Heartbeat & Task Execution
OBJECTIVE	Allow a registered agent to retrieve and execute pending deployment tasks.
PRIMARY ACTOR	Agent Daemon (automated process on the client machine).
SECONDARY ACTOR	System (Express API, File System, SQLite database).
PRE-CONDITION	Agent is registered and holds a valid apiKey.
NOMINAL SCENARIO	1. The agent polls every 30 seconds with a POST <code>/api/agents/heartbeat</code> call including apiKey. 2. The system updates the client's last seen time and status to online. 3. The system searches for any DeploymentClients that have been assigned to this particular client. 4. The system responds with tasks containing work that needs doing. 5. For each of the tasks, the agent calls status downloading and downloads the package file. 6. The agent unzips the ZIP file and runs <code>install.sh</code> if there is one. 7. The agent calls status installing and either succeeds or fails. 8. The system updates the DeploymentClient entry, aggregates the counters within the Deployment object, and creates an InstalledPackage object if successful.
ALTERNATIVE SCENARIO	5.1) In case the file does not get downloaded successfully (file not found on the server side), the agent will report back failed with an errorMessage. 6.1) In case <code>install.sh</code> has returned a failure (non-zero exit code), the agent will report back a failure. 1.1) In case the server is unreachable, the agent will retry during the next poll cycle.

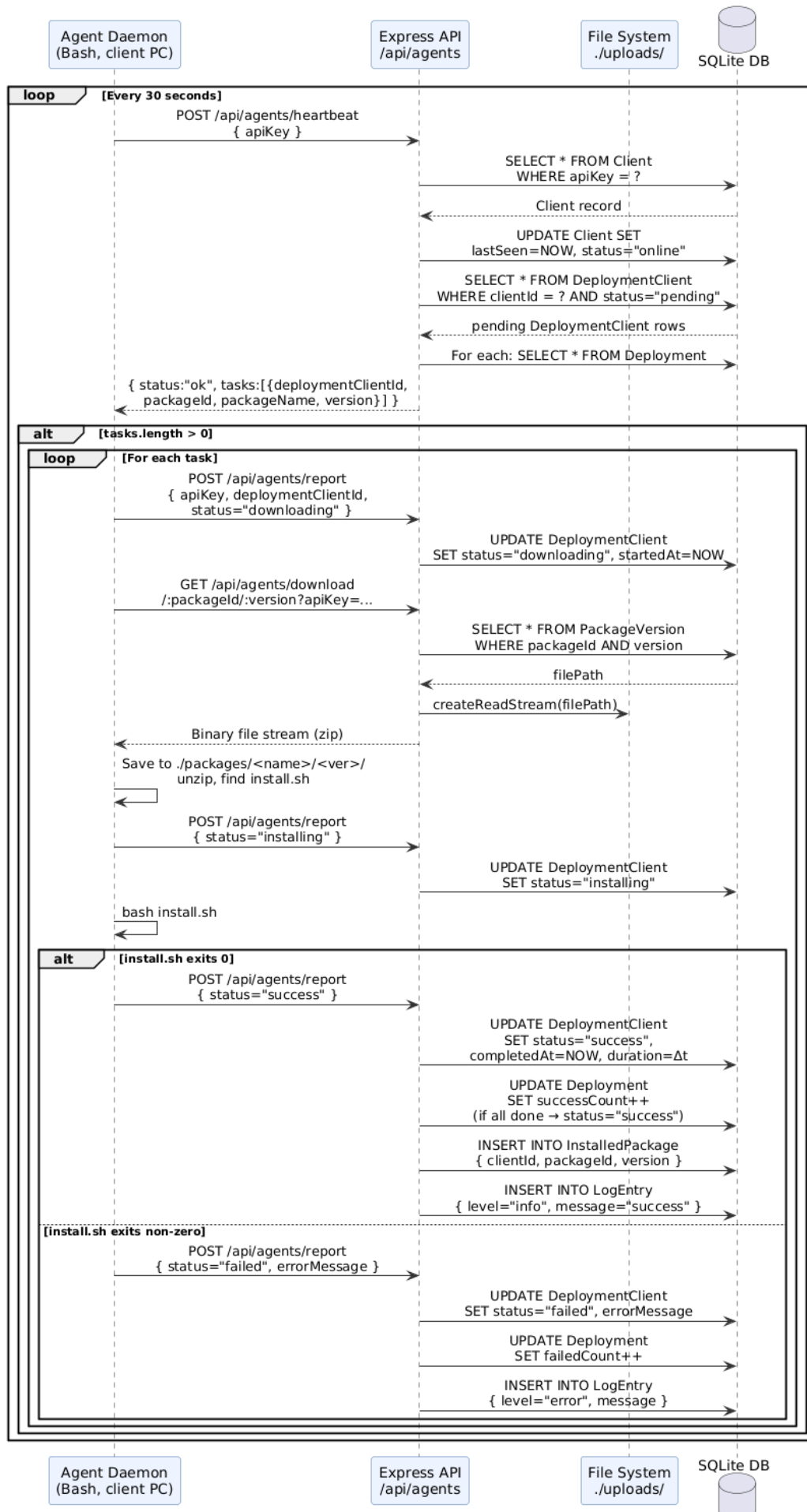


Figure 2.5: Sequence Diagram — Agent Heartbeat & Task Execution

2.4.6 Agent Registration

Table 2.6: Textual Description — Agent Registration

USE CASE	Agent Registration
OBJECTIVE	Register a new client machine with the platform so it can receive deployments.
PRIMARY ACTOR	IT Technician (runs the installer on the client machine).
SECONDARY ACTOR	System (Express API, SQLite database, local file system of the client machine).
PRE-CONDITION	The backend server is reachable from the client machine on the local network.
NOMINAL SCENARIO	1. The IT technician executes the <code>install.sh</code> command on the client system, giving the server IP address. 2. The installer retrieves the hostname, IP address, OS type, and CPU architecture of the machine. 3. The installer POSTs <code>/api/agents/register</code> to the server. 4. The server creates a new Client entry and creates a UUID apiKey. 5. The installer stores the configuration (SERVER_URL, CLIENT_ID, API_KEY) into <code>/opt/download-center-agent/agent.conf</code>. 6. The installer creates the agent daemon script and systemd service. 7. The agent becomes operational in the background.
ALTERNATIVE SCENARIO	3.1) In case the hostname is present in the database, the server updates the client information and returns the same apiKey (re-registration case). 3.2) If the server cannot be reached, an error message is displayed, and the installer quits.

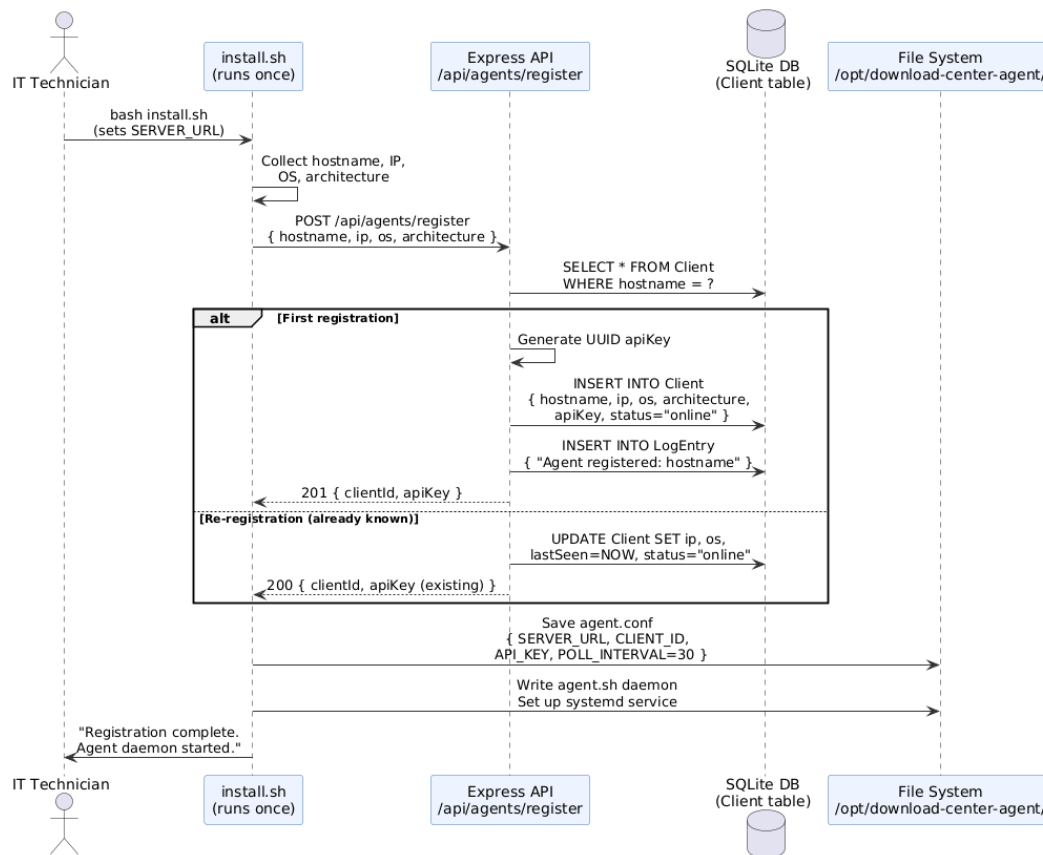


Figure 2.6: Sequence Diagram — Agent Registration

2.5 Class Diagram

2.5.1 Definition

Class Diagrams represent graphically the internal structure of the system, consisting of classes (attributes + operations). Using a class diagram, we can:

- Specify the future elements of the future system.
- Properly organize development.
- Separate elements to divide development among developers.
- Create the system properly.

Two class diagrams have been created. The first one lacks attributes in order to find out what classes there are and what relations exist between those classes. In the second class diagram, each class has been described with its attributes and methods.

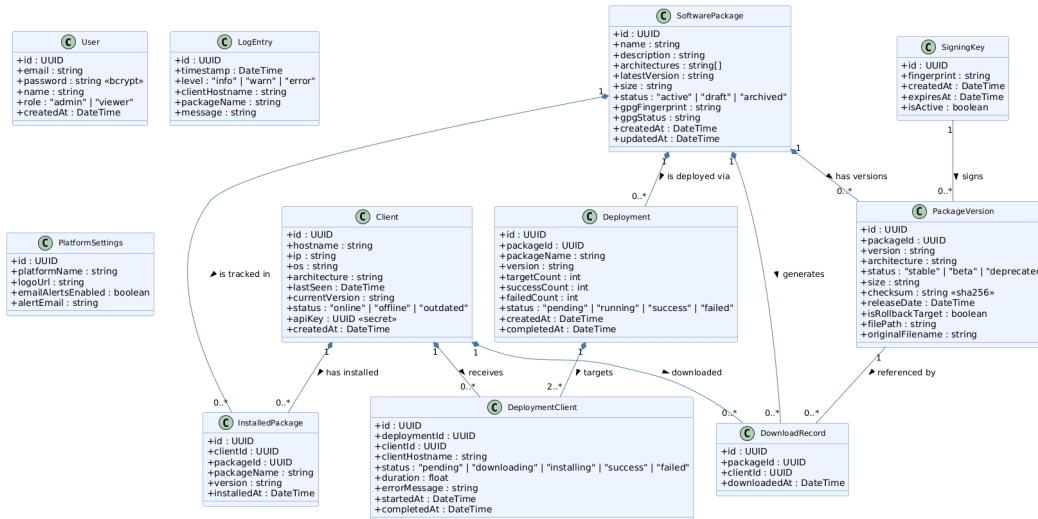


Figure 2.7: Class Diagram of the Automated Download Platform

2.6 Relational Model

The relational model is based on the class diagram where each entity is translated into a table within a SQLite database using TypeORM. The primary key for tables is represented by UUID created on the server side. Foreign key references between tables are set via columns with names rather than using TypeORM JoinColumn annotations.

User	(id, email, password, name, role, createdAt)
SoftwarePackage	(id, name, description, architectures, latestVersion, size, status, gpgFingerprint, gpgStatus, createdAt, updatedAt)
PackageVersion	(id, packageId*, version, architecture, status, size, checksum, releaseDate, isRollbackTarget, filePath, originalFilename)
Client	(id, hostname, ip, os, architecture, lastSeen, currentVersion, status, apiKey, createdAt)
Deployment	(id, packageId*, packageName, version, targetCount, successCount, failedCount, status, createdAt, completedAt)
DeploymentClient	(id, deploymentId*, clientId*, clientHostname, status, duration, errorMessage, startedAt, completedAt)
InstalledPackage	(id, clientId*, packageId*, packageName, version, installedAt)

LogEntry	(id, timestamp, level, clientHostname, packageName, message)
DownloadRecord	(id, packageId*, clientId*, downloadedAt)
SigningKey	(id, fingerprint, createdAt, expiresAt, isActive)
PlatformSettings	(id, platformName, logoUrl, emailAlertsEnabled, alertEmail)

** denotes a foreign key reference.*

2.7 Conclusion

In this chapter, we have achieved the final stage of designing our Automated Download Platform using the UML method. This is a crucial aspect of our work that helps us gain a thorough understanding of how the proposed system works. Most importantly, it equips us to implement the system, which will be discussed in the next chapter.

Chapter 3

Implementation

3.1 Introduction

After completing the design and conceptual study of the Download Center Platform, this chapter presents the implementation and deployment of the system. We begin by introducing the development environment, the tools, technologies, and programming languages used. We then present the different interfaces and functionalities of the application, illustrated with screenshots, following the flow described in the architecture documentation.

3.2 Development Environment

3.2.1 Programming Languages

3.2.1.1 TypeScript / JavaScript

TypeScript is a strongly typed, open-source programming language developed by Microsoft. It is a superset of JavaScript that adds optional static typing and class-based object-oriented programming. TypeScript compiles down to plain JavaScript and runs in any browser or JavaScript engine. In this project, TypeScript is used for both the backend (Express API) and the frontend (React application), providing better code reliability, autocompletion, and easier refactoring throughout the codebase.

3.2.1.2 Bash (Shell Scripting)

Bash (Bourne Again SHell) is a Unix shell and command language widely used for system administration and automation on Linux systems. In this project, Bash is the language used to implement the Agent Daemon — the lightweight background process that runs on each client machine. The agent is responsible for registering with the server, polling for pending deployment tasks, downloading software packages, executing installation scripts, and reporting back the deployment status.

3.2.1.3 SQL (SQLite)

SQL (Structured Query Language) is the standard language for managing relational databases. SQLite is a lightweight, serverless, file-based relational database engine. In this project, all platform data — users, software packages, versions, clients, deployments, logs, and settings — is stored in a single local SQLite file (`./data/database.sqlite`), managed through the TypeORM ORM layer. This choice provides simplicity, portability, and zero-configuration deployment for the backend.

3.2.2 Frameworks and Libraries

3.2.2.1 React + Vite

React is an open-source JavaScript library developed by Meta for building user interfaces, based on a component model and a virtual DOM for efficient rendering. Vite is a modern, lightning-fast frontend build tool that provides an optimized development server with hot module replacement. Together, React and Vite form the foundation of the Web Application (Admin Dashboard), running on port 5173 and communicating with the backend via Axios through Vite's built-in proxy.

3.2.2.2 Express.js

Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building RESTful APIs. It handles HTTP routing, middleware execution, request parsing, and response formatting. In this project, Express is the core of the Backend API (running on port 3000), managing all routes for authentication, package uploads, deployment orchestration, agent communication, and log management.

3.2.2.3 TypeORM

TypeORM is an Object-Relational Mapping (ORM) framework for TypeScript and JavaScript that supports various database engines including SQLite, PostgreSQL,^[5] and MySQL. It enables developers to define database entities as TypeScript classes and interact with the database through a high-level API rather than writing raw SQL queries. TypeORM manages the data models (User, SoftwarePackage, PackageVersion, Client, Deployment, DeploymentClient, InstalledPackage, LogEntry, etc.) and handles all database migrations and schema synchronization.

3.2.3 Development Tools

3.2.3.1 Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It allows JavaScript to run on the server side, enabling the development of fast and scalable network applications. Node.js is the runtime on which both the Express backend and the build toolchain (Vite, TypeScript compiler) execute. Its non-blocking, event-driven architecture makes it well-suited for handling concurrent API requests from multiple agents polling simultaneously.

3.2.3.2 Visual Studio Code

Visual Studio Code (VS Code) is a lightweight, open-source code editor developed by Microsoft, widely adopted for web and backend development. It provides rich support for TypeScript and JavaScript through IntelliSense, integrated debugging, Git version control, and an extensive ecosystem of extensions. VS Code was used as the primary IDE throughout the development of the Download Center Platform.

3.2.3.3 Additional Libraries

The following additional libraries were used in the project:

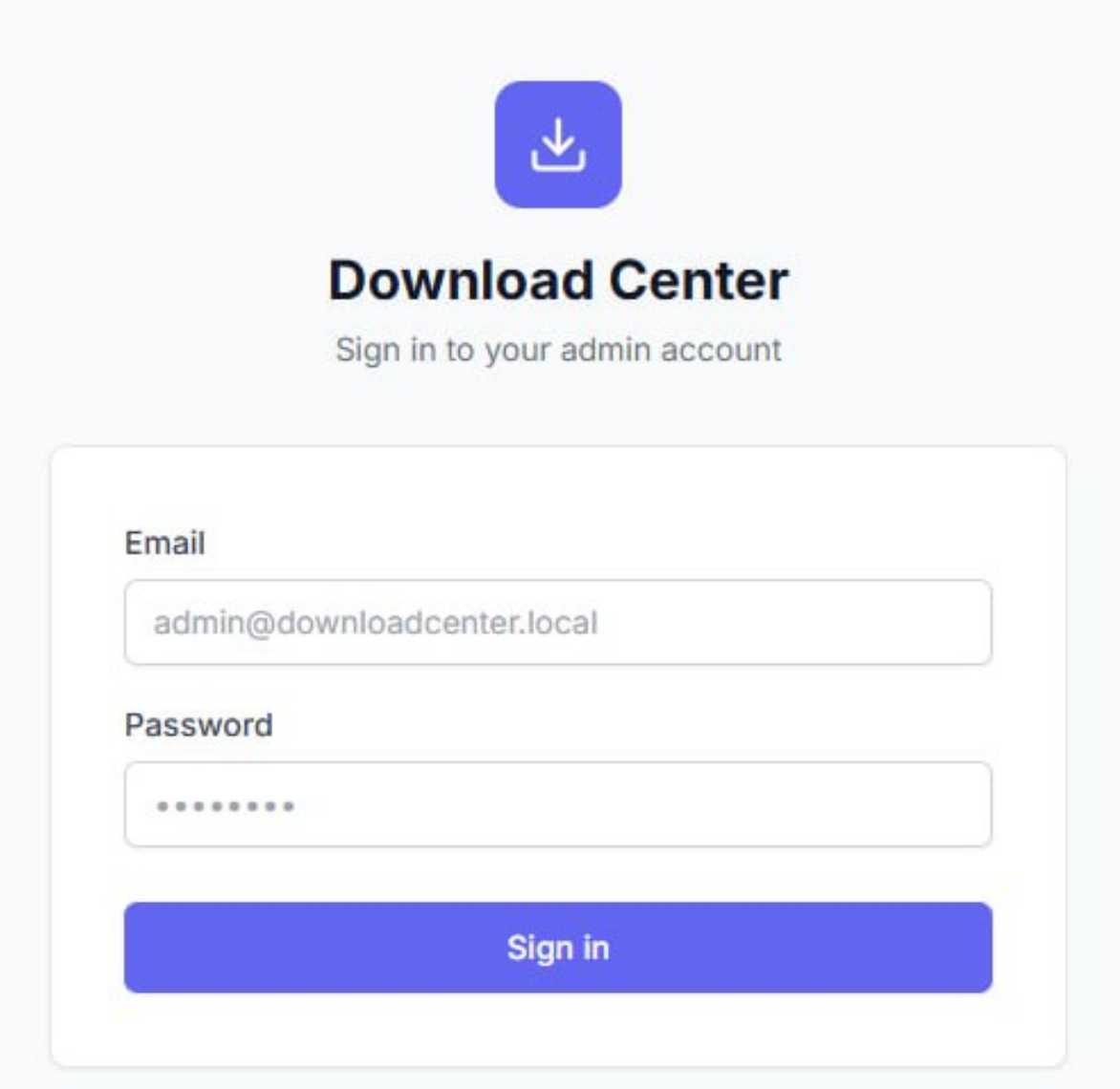
- **JSON Web Token (JWT)** — for stateless authentication and secure session management between the admin dashboard and the backend API.
- **Multer** — a Node.js middleware for handling multipart/form-data, used for uploading software package ZIP files to the server.
- **bcrypt** — for hashing and verifying administrator passwords securely in the database.
- **Axios** — a promise-based HTTP client used by the React frontend to communicate with the backend REST API.[7]
- **systemd** — used on Linux client machines to register the agent daemon as a background service that starts automatically on boot.

3.3 Application Interfaces

3.3.1 Authentication Interface

The platform is secured by a JWT-based authentication system. The administrator accesses the login page and provides their email address and password. Upon successful

validation, the server generates a signed JWT token valid for 24 hours, which is stored in the application's memory and attached to all subsequent API requests.



The image shows a web interface for a 'Download Center'. At the top, there is a blue square icon with a white download arrow. Below the icon, the text 'Download Center' is displayed in a large, bold, black font. Underneath this, the text 'Sign in to your admin account' is shown in a smaller, gray font. The main part of the interface is a white rounded rectangle containing two input fields. The first field is labeled 'Email' and contains the text 'admin@downloadcenter.local'. The second field is labeled 'Password' and contains a series of dots, indicating a masked password. Below these fields is a large blue button with the text 'Sign in' in white.

Figure 3.1: Authentication Interface

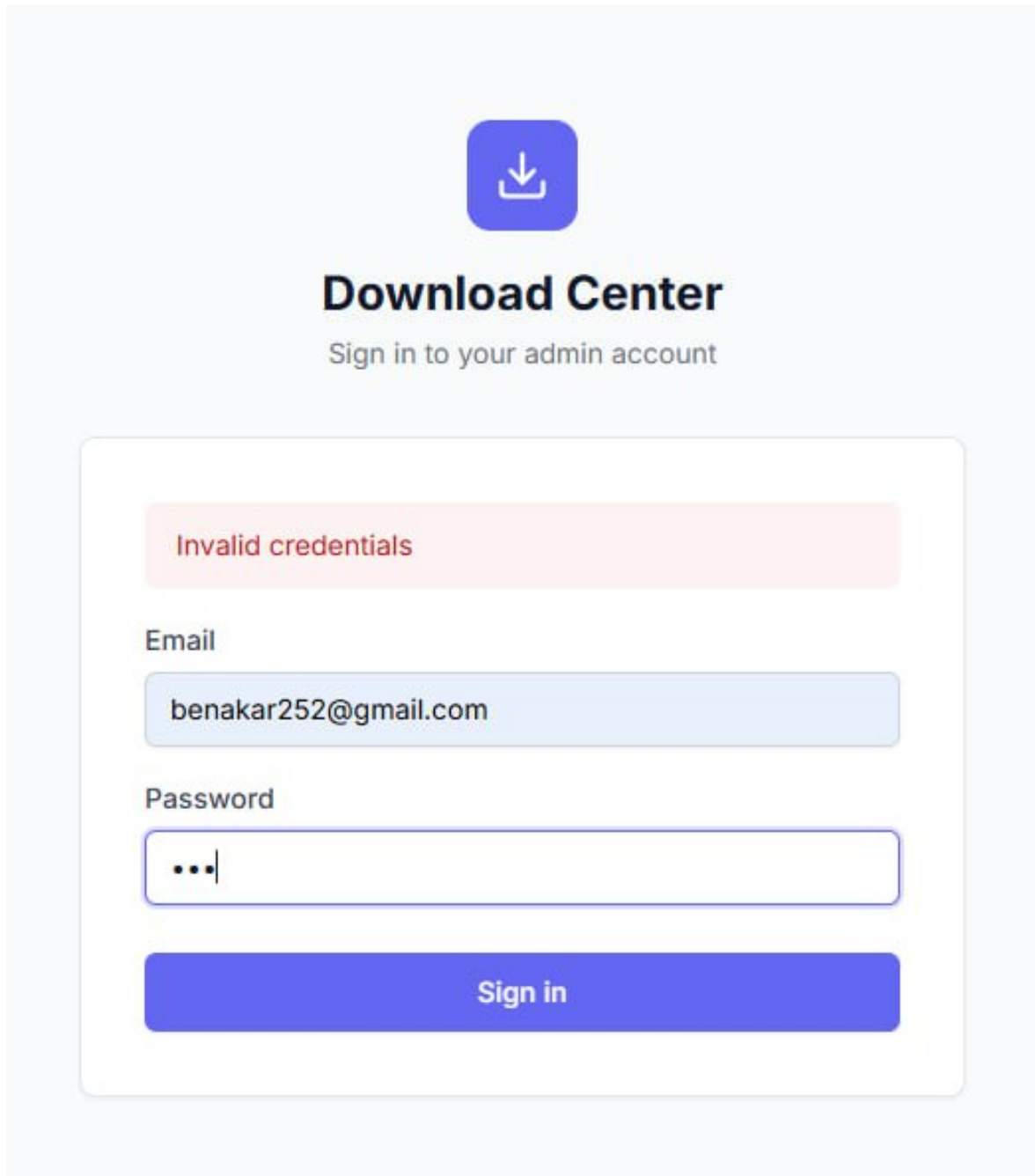


Figure 3.2: Invalid Authentication — Error Message

3.3.2 Main Dashboard

After logging in, the administrator is redirected to the main dashboard. This interface provides a global overview of the platform's activity, including the number of registered client machines, total software packages, recent deployments, and success/failure statistics. Key metrics are displayed in summary cards, and recent activity is visible in the activity feed.

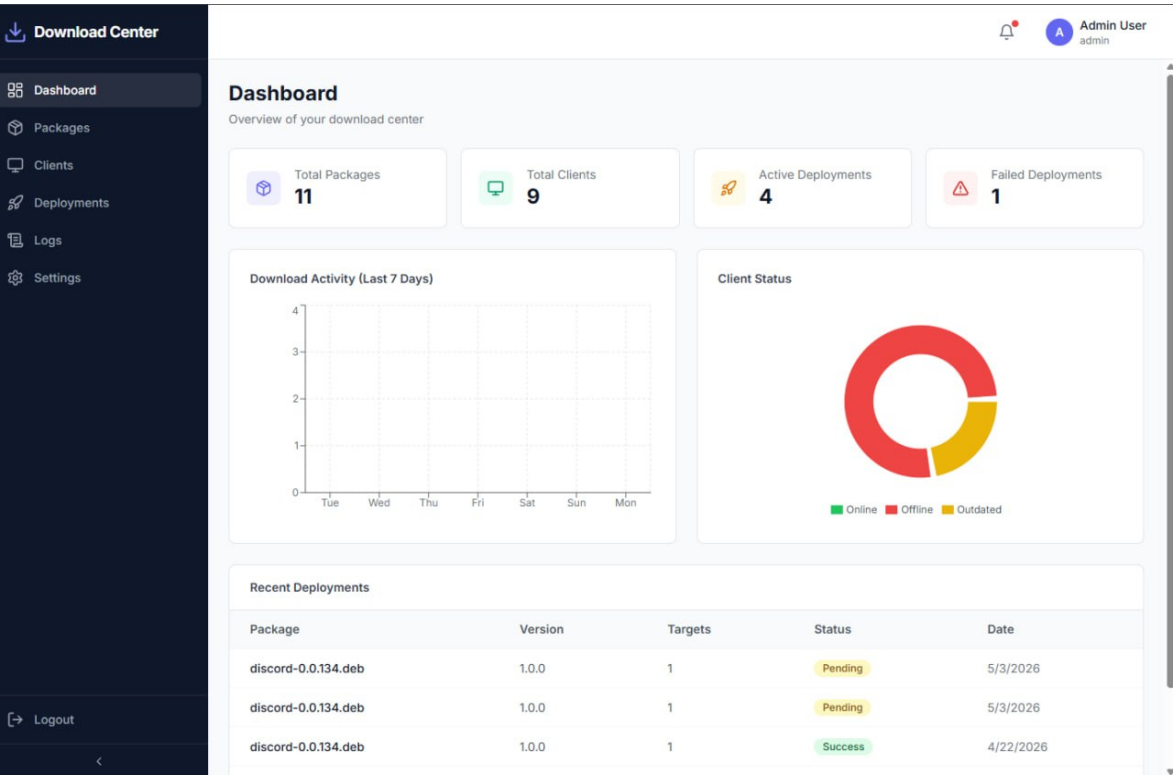
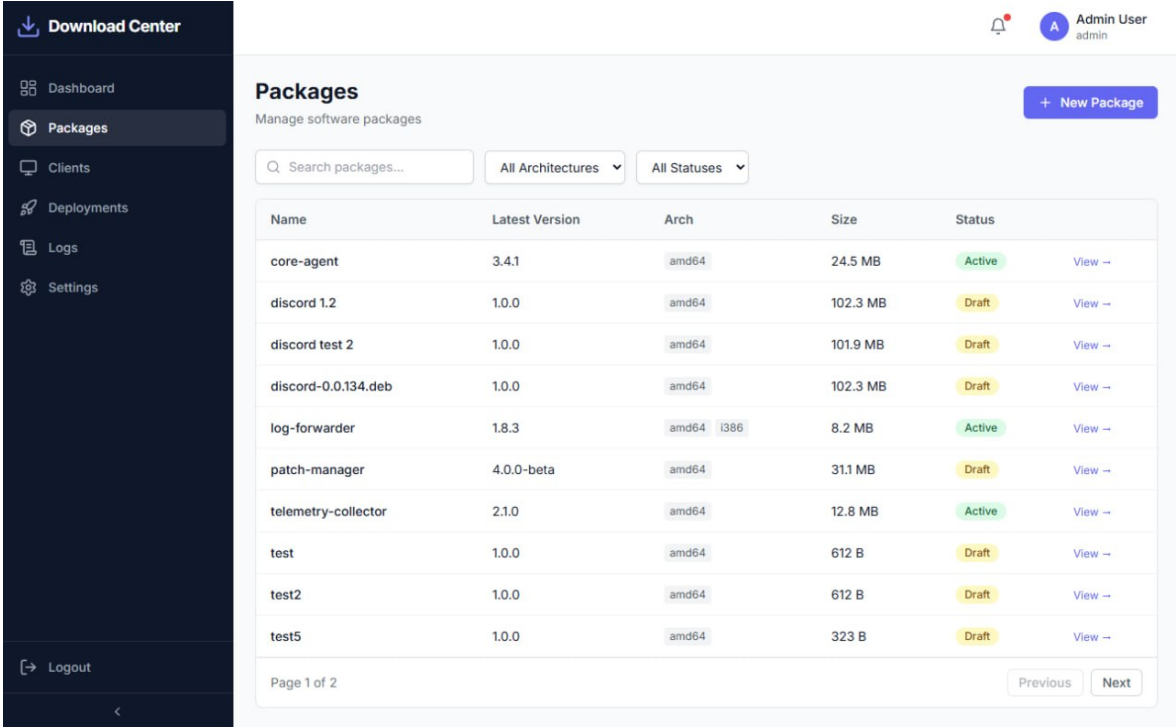


Figure 3.3: Main Dashboard Interface

3.3.3 Package Management Interface

3.3.3.1 Package List

The package management section allows the administrator to browse all uploaded software packages. Each entry displays the package name, latest version, supported architectures, current status (active, draft, or archived), and the date of last update. A search and filter bar enables quick navigation through the catalog.



The screenshot displays the 'Download Center' interface. On the left is a dark sidebar with navigation links: Dashboard, Packages (selected), Clients, Deployments, Logs, and Settings. At the bottom of the sidebar is a 'Logout' button. The main content area is titled 'Packages' with the subtitle 'Manage software packages'. In the top right corner, there is a notification bell and a user profile for 'Admin User' (admin). A '+ New Package' button is located in the top right of the main area. Below the title, there is a search bar labeled 'Search packages...' and two dropdown menus for 'All Architectures' and 'All Statuses'. The central part of the interface is a table listing software packages. The table has columns for Name, Latest Version, Arch, Size, and Status. The status column uses color-coded labels: 'Active' in green and 'Draft' in yellow. Each row includes a 'View' link with a right-pointing arrow. The table lists 11 packages, including 'core-agent', 'discord 1.2', 'discord test 2', 'discord-0.0.134.deb', 'log-forwarder', 'patch-manager', 'telemetry-collector', and several test packages. The bottom of the table shows 'Page 1 of 2' and 'Previous'/'Next' navigation buttons.

Name	Latest Version	Arch	Size	Status
core-agent	3.4.1	amd64	24.5 MB	Active
discord 1.2	1.0.0	amd64	102.3 MB	Draft
discord test 2	1.0.0	amd64	101.9 MB	Draft
discord-0.0.134.deb	1.0.0	amd64	102.3 MB	Draft
log-forwarder	1.8.3	amd64 i386	8.2 MB	Active
patch-manager	4.0.0-beta	amd64	31.1 MB	Draft
telemetry-collector	2.1.0	amd64	12.8 MB	Active
test	1.0.0	amd64	612 B	Draft
test2	1.0.0	amd64	612 B	Draft
test5	1.0.0	amd64	323 B	Draft

Figure 3.4: Package Management — Package List

3.3.3.2 Upload a New Package

The administrator can upload a new software package by filling in the package name, version number, supported architecture, and an optional description, then attaching a ZIP file. Upon submission, the backend computes a SHA256 checksum of the file for integrity verification, saves the file to the uploads directory, and creates the corresponding database records (SoftwarePackage and PackageVersion).

The screenshot shows a web application interface for a 'Download Center'. On the left is a dark sidebar with navigation links: 'Dashboard', 'Packages' (highlighted), 'Clients', 'Deployments', 'Logs', and 'Settings'. At the bottom of the sidebar is a 'Logout' button. The main content area is titled 'New Package' with the subtitle 'Create a new software package'. In the top right corner, there is a notification bell icon and a user profile for 'Admin User' (admin). The form itself contains several fields: a 'Package Name' text input with the placeholder 'e.g. core-agent'; a 'Description' text area with the placeholder 'Brief description of the package'; an 'Architectures' section with two buttons labeled 'amd64' and 'i386'; and a 'Package File' section featuring a large dashed box with an upload icon and the text 'Click to upload or drag & drop' and 'Accepted: .deb, .pkg, .tar, .zst'. At the bottom right of the form are 'Cancel' and 'Create Package' buttons.

Figure 3.5: New Package Upload Form

3.3.3.3 Package Details

Clicking on a package opens its detail view, which lists all available versions with their release dates, architecture, file size, checksum, and stability status (stable, beta, or deprecated). From this view, the administrator can also create a new deployment targeting that package.

Name	Latest Version	Arch	Size	Status
core-agent	3.4.1	amd64	24.5 MB	Active
discord 1.2	1.0.0	amd64	102.3 MB	Draft
discord test 2	1.0.0	amd64	101.9 MB	Draft
discord-0.0.134.deb	1.0.0	amd64	102.3 MB	Draft
log-forwarder	1.8.3	amd64 i386	8.2 MB	Active
patch-manager	4.0.0-beta	amd64	31.1 MB	Draft
telemetry-collector	2.1.0	amd64	12.8 MB	Active
test	1.0.0	amd64	612 B	Draft
test2	1.0.0	amd64	612 B	Draft
test5	1.0.0	amd64	323 B	Draft

Figure 3.6: Package Detail View with Version History

3.3.4 Deployment Management Interface

3.3.4.1 Create a Deployment

To deploy a software package, the administrator selects the desired package and version, then chooses one or more target client machines from the registered clients list. Upon confirmation, the backend creates a Deployment record and one DeploymentClient row per target with an initial status of pending. From this point, the selected agents will receive the task during their next heartbeat poll (every 30 seconds).

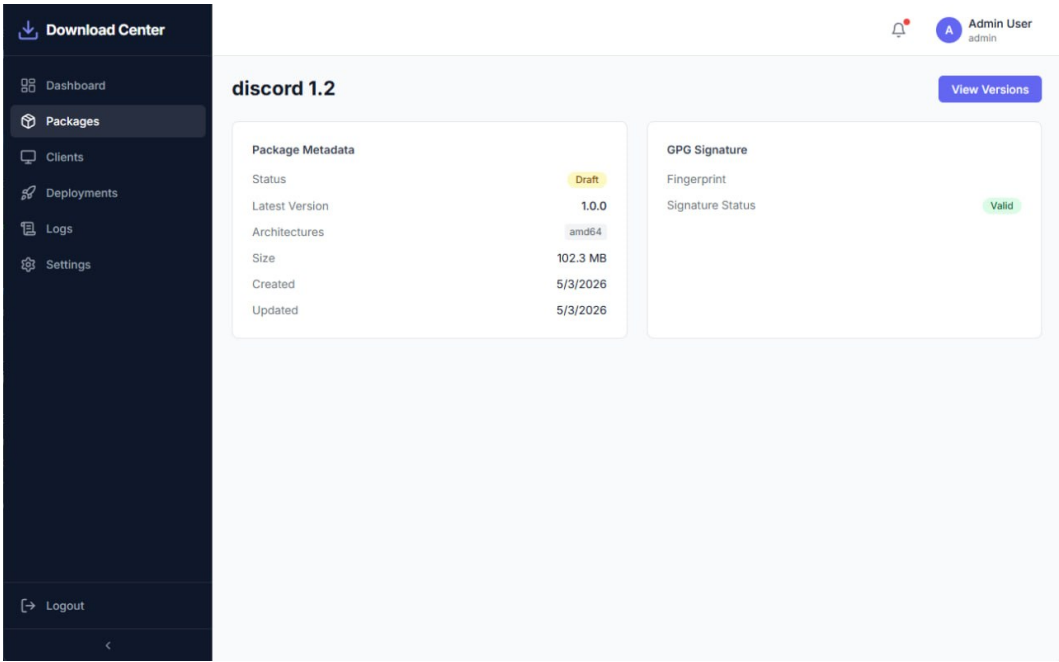


Figure 3.7: Deployment Form — Select Package

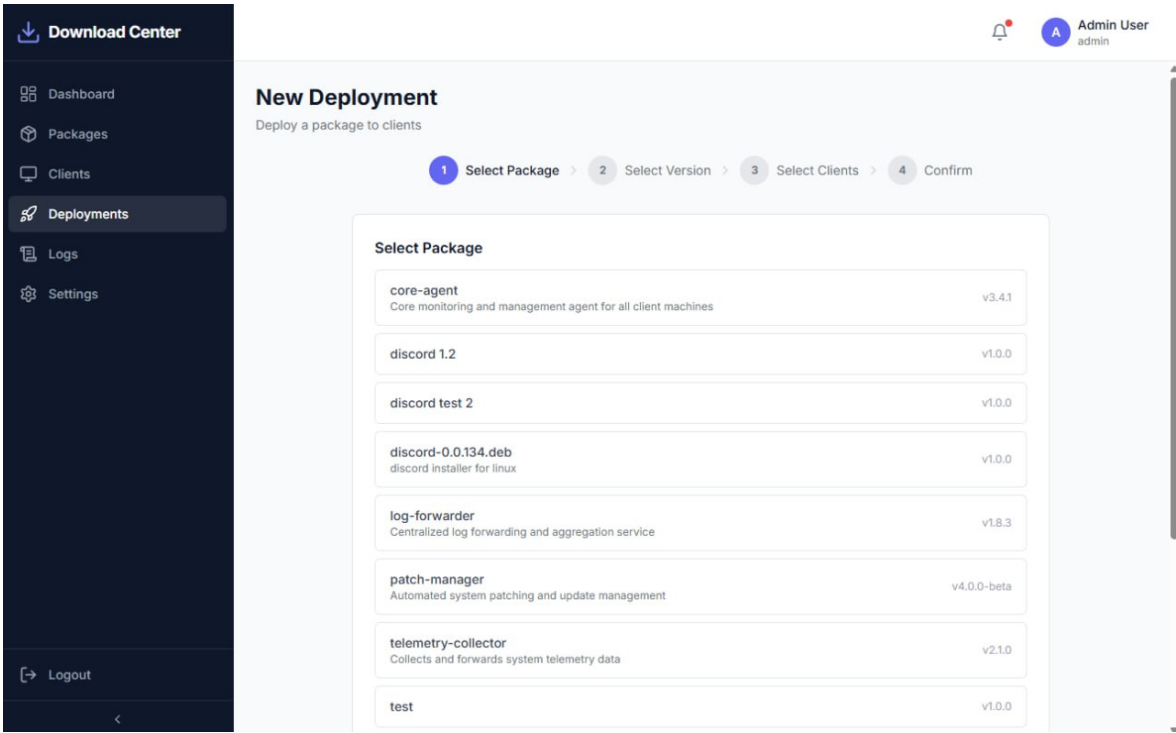


Figure 3.8: Deployment Form — Select Version

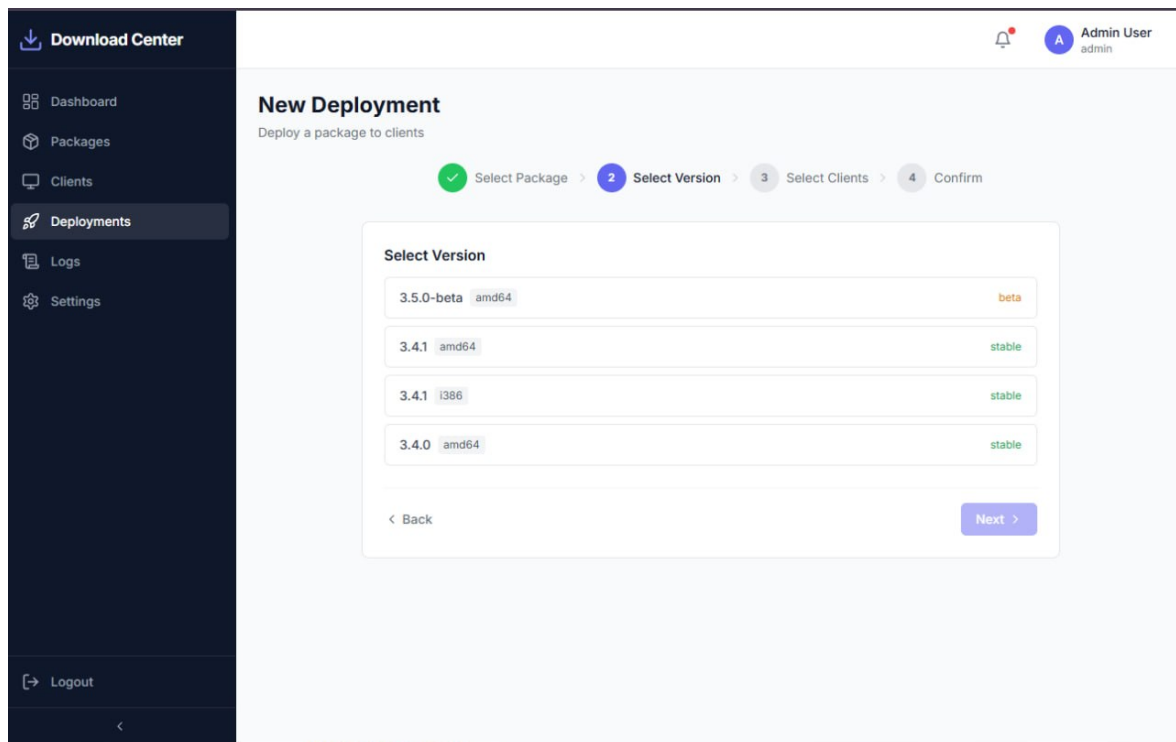


Figure 3.9: Deployment Form — Select Client

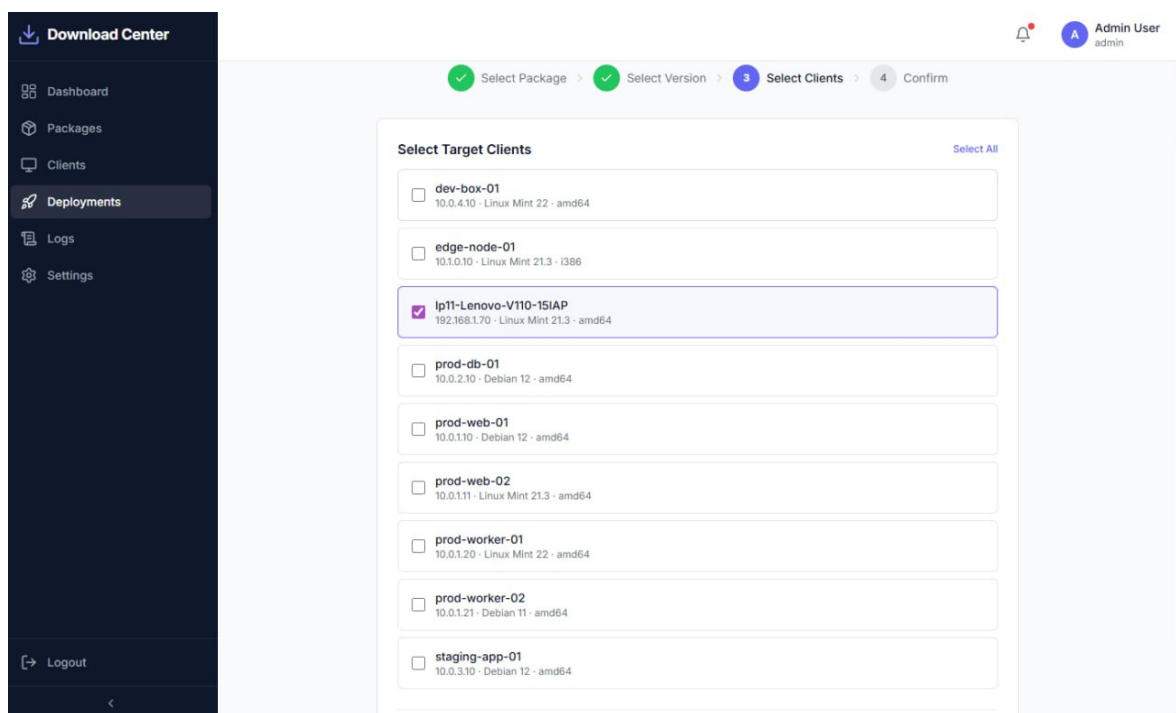


Figure 3.10: Deployment Form — Confirmation

3.3.4.2 Monitor Deployment Progress

The deployment monitoring view displays real-time progress for each targeted client machine. As the agent daemon executes the task, it reports its status at each step:

downloading, installing, success, or failed. The interface updates these statuses live. A progress counter at the top shows how many clients have succeeded, are in progress, or have failed. In case of failure, the error message reported by the agent is displayed alongside the client entry.

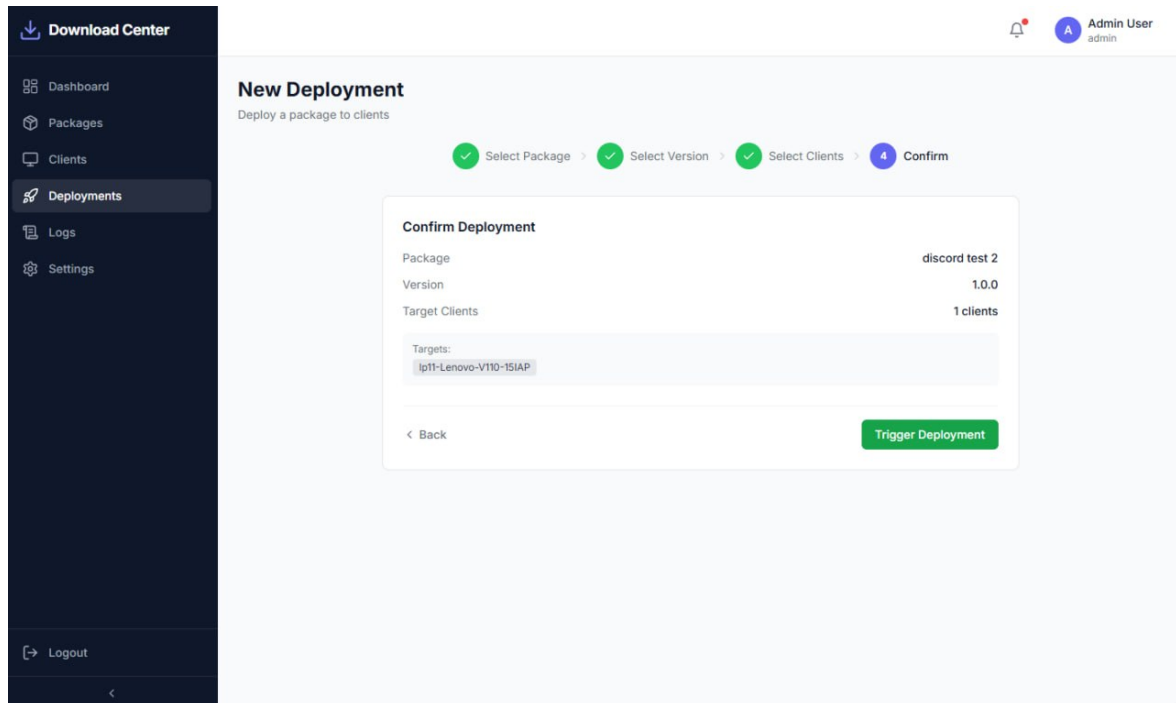
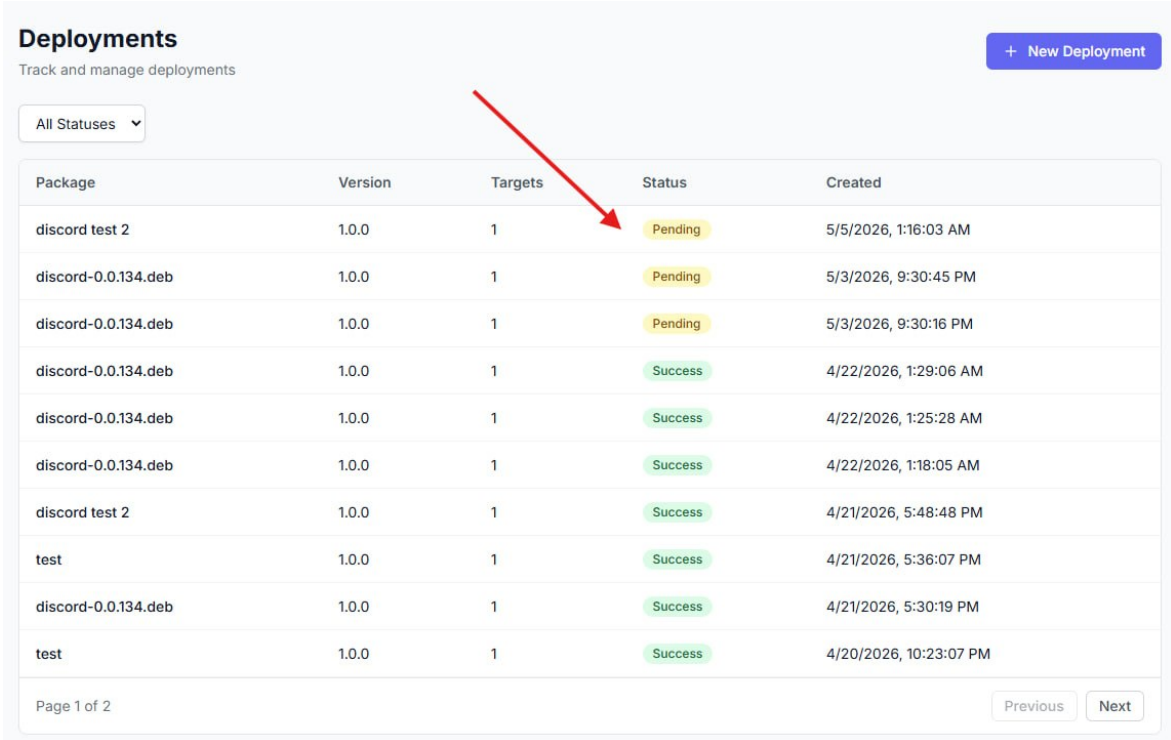


Figure 3.11: Deployment Monitor — Per-Client Status

3.3.5 Client Management Interface

3.3.5.1 Registered Clients List

The clients section lists all machines that have registered their agent with the server. Each entry displays the client's hostname, IP address, operating system, architecture, current status (online, offline, or outdated), and the timestamp of its last heartbeat. A background heartbeat monitor runs every 60 seconds on the server and automatically marks clients as offline if they have not checked in within the expected interval.



Deployments
Track and manage deployments

+ New Deployment

All Statuses ▾

Package	Version	Targets	Status	Created
discord test 2	1.0.0	1	Pending	5/5/2026, 1:16:03 AM
discord-0.0.134.deb	1.0.0	1	Pending	5/3/2026, 9:30:45 PM
discord-0.0.134.deb	1.0.0	1	Pending	5/3/2026, 9:30:16 PM
discord-0.0.134.deb	1.0.0	1	Success	4/22/2026, 1:29:06 AM
discord-0.0.134.deb	1.0.0	1	Success	4/22/2026, 1:25:28 AM
discord-0.0.134.deb	1.0.0	1	Success	4/22/2026, 1:18:05 AM
discord test 2	1.0.0	1	Success	4/21/2026, 5:48:48 PM
test	1.0.0	1	Success	4/21/2026, 5:36:07 PM
discord-0.0.134.deb	1.0.0	1	Success	4/21/2026, 5:30:19 PM
test	1.0.0	1	Success	4/20/2026, 10:23:07 PM

Page 1 of 2

Previous Next

Figure 3.12: Registered Client Machines Interface

3.3.5.2 Client Detail View

Clicking on a client opens its detail page, which shows the full registration information, current agent version, and the list of all software packages installed on that machine, along with their installation dates and versions. This provides the administrator with a complete software inventory per machine.

Clients
Connected client machines

Q Search clients...

Hostname	IP Address	OS	Arch	Last Seen	Version	Status
dev-box-01	10.0.4.10	Linux Mint 22	amd64	4/16/2026, 2:04:34 PM	3.3.0	Outdated
edge-node-01	10.1.0.10	Linux Mint 21.3	i386	4/16/2026, 2:14:34 AM	3.3.0	Offline
lp11-Lenovo-V110-15IAP	192.168.1.70	Linux Mint 21.3	amd64	4/22/2026, 1:34:31 AM		Offline
prod-db-01	10.0.2.10	Debian 12	amd64	4/16/2026, 2:11:34 PM	3.4.0	Offline
prod-web-01	10.0.1.10	Debian 12	amd64	4/16/2026, 2:12:34 PM	3.4.1	Offline
prod-web-02	10.0.1.11	Linux Mint 21.3	amd64	4/16/2026, 2:13:34 PM	3.4.1	Offline
prod-worker-01	10.0.1.20	Linux Mint 22	amd64	4/16/2026, 2:10:34 PM	3.3.0	Outdated
prod-worker-02	10.0.1.21	Debian 11	amd64	4/16/2026, 2:12:34 PM	3.4.1	Offline
staging-app-01	10.0.3.10	Debian 12	amd64	4/16/2026, 2:09:34 PM	3.4.1	Offline

Figure 3.13: Client Detail — Installed Packages

3.3.6 Agent Registration (First-Time Setup)

The agent is installed on each client machine by an IT technician running the provided `install.sh` script. The script automatically collects the machine's hostname, IP address, operating system, and architecture, then sends a registration request to the backend API. If the machine is registering for the first time, the server generates a unique API key and returns it to the installer. The script saves the configuration (`SERVER_URL`, `CLIENT_ID`, `API_KEY`, `POLL_INTERVAL`) to a local file and registers the agent as a systemd service, which will start automatically on every reboot.

```
lp11@lp11-Lenovo-V110-15IAP:~/Desktop/secpfe$ bash /opt/download-center-agent/agent.sh
[2026-05-05 16:28:30] Agent started. Polling every 30s...
[2026-05-05 16:28:30] Server: http://192.168.1.64:3000
[2026-05-05 16:28:30] Found 3 pending task(s)
[2026-05-05 16:28:31] 📦 New task: discord-0.0.134.deb v1.0.0
[2026-05-05 16:28:31]   Downloading...
[2026-05-05 16:28:46]   ✓ Downloaded: discord-0.0.134.deb
[2026-05-05 16:28:46]   Installing (detected type: .deb)...
[2026-05-05 16:28:46]   Installing .deb with dpkg: /opt/download-center-agent/packages/discord-0.0.134.deb/1.0.0/discord-0.0.134.deb
[2026-05-05 16:28:54]   ✓ .deb installed successfully
[2026-05-05 16:28:54]   Done: discord-0.0.134.deb v1.0.0
```

Figure 3.14: Agent Registration — Terminal Output

Once registered, the agent daemon runs continuously in the background. Every 30 seconds it sends a heartbeat to the server. If the server has pending deployment tasks

for that client, they are returned in the response and the agent immediately begins execution: downloading the ZIP file, extracting it, running the `install.sh` script inside the package, and reporting the outcome back to the server.

3.4 Conclusion

At the end of this chapter, the implementation phase of the Download Center Platform has been fully described. The combination of a React/Vite frontend, an Express/TypeORM backend, and a Bash agent daemon allowed us to build a complete, three-tier software distribution system. The choice of lightweight and proven technologies ensured that the platform remains maintainable, extensible, and deployable without complex infrastructure. All features described in the architecture documentation — package uploads, version management, deployment orchestration, real-time status tracking, and client management — have been developed and validated. The system is ready for use within Algérie Télécom’s internal network environment.

General Conclusion

In fact, this project managed to solve the real problem of operations for Algeria Telecom by developing a three-tier working software distribution system. A combination of the React front-end, Express and TypeORM back-end, and a bash daemon led to development of the system capable of automating the deployment process, ensuring integrity of packages, and offering full visibility over each client machine to the administrator.

On top of that, from the perspective of the learning process, this project provided useful practical skills in the area of software engineering, including system analysis, UML design, full-stack programming, and Linux system administration. Potential improvements may include supporting Windows clients, implementing GPG signatures,[14] applying role-based security, and switching to the PostgreSQL database., [15]

References / Bibliography

I. Institutional & Legal Sources

- [1] République Algérienne Démocratique et Populaire, *Loi n° 2000-03 du 5 août 2000 fixant les règles générales relatives à la poste et aux télécommunications*, Journal Officiel de la République Algérienne, no. 48, 2000. [Online]. Available: <https://www.arpt.dz/>
- [2] Autorité de Régulation de la Poste et des Télécommunications (ARPT), *Rapports annuels sur le marché des télécommunications en Algérie*, ARPT, Algiers, 2003–2023. [Online]. Available: <https://www.arpt.dz/fr/doc/pub/rap/>
- [3] Algérie Télécom, *Présentation institutionnelle et offres de services*, Algérie Télécom S.P.A., Algiers, 2023. [Online]. Available: <https://www.algeriatelecom.dz/>
- [4] Ministère de la Poste, des Télécommunications, des Technologies et du Numérique (MPTIC), *Stratégie nationale de développement du numérique*, MPTIC, Algiers, 2020. [Online]. Available: <https://www.mpttn.gov.dz/>

II. Software Engineering & Architecture

- [5] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, Hoboken, NJ, 2016.
- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, Reading, MA, 1994.
- [7] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

III. Software Package Management & Update Systems

- [8] J. Cappos, J. Samuel, S. Baker, and J. H. Hartman, “A Look in the Mirror: Attacks on Package Managers,” in *Proc. 15th ACM Conf. on Computer and*

- Communications Security (CCS)*, Alexandria, VA, 2008, pp. 565–574.
- [9] K. R. B. Butler, S. McLaughlin, and P. D. McDaniel, “Rootkit-Resistant Disks,” in *Proc. 15th ACM Conf. on Computer and Communications Security*, 2008.
- [10] The Update Framework (TUF) Authors,[8] *The Update Framework Specification*, Version 1.0, 2023. [Online]. Available: <https://theupdateframework.io/>
- [11] Microsoft Corporation, *Windows Server Update Services (WSUS) Documentation*, Microsoft Docs, Redmond[11], WA, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/windows-server/administration/windows-server-update->
- [12] Canonical Ltd., *Ubuntu APT Package Management System Documentation*, 2023. [Online]. Available: <https://help.ubuntu.com/community/AptGet/Howto>

IV. Digital Signatures & Cryptographic Security

- [13] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson Education, Hoboken, NJ, 2017.
- [14] National Institute of Standards and Technology (NIST), *Digital Signature Standard (DSS)*, FIPS PUB 186-5, NIST, Gaithersburg, MD, 2023. [Online]. Available: <https://csrc.nist.gov/publications/detail/fips/186/5/final>
- [15] D. Cooper et al., *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, RFC 5280, IETF, 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5280>

V. Client–Server Architecture & Distributed Systems [18]

- [16] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. CreateSpace, 2017.
- [17] M. T. Jones, “Anatomy of a Software Update System,[17] ” IBM developerWorks, IBM Corporation, 2008. [Online]. Available: <https://developer.ibm.com/>
- [18] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, *Distributed Systems: Concepts and Design*, 5th ed. Addison-Wesley, Harlow, UK, 2011.